

Python Fluency Reference

Everything you need to turn an English requirement into working Python

This is a **reference**, not a lesson. Read a section, then close it and write code. Understanding a page here is worth nothing until you can write the pattern from a blank file. During a timed drill, answer from memory first; look things up only after you try.

Part	Contents
1	How to attack any problem (decomposition + tool picker)
2	Program skeleton, input, output, f-strings
3	Variables, numbers, arithmetic, comparisons, booleans
4	Strings
5	Conditions: if / elif / else (and ordering rules)
6	Loops: for, while, range, break, continue
7	The 12 loop patterns (accumulator, counter, max, filter, search ...)
8	Lists (everything)
9	Dictionaries
10	Sets and tuples
11	Functions
12	Pricing / rules-to-code patterns (pizza, parking, tax ...)
13	Testing your code
14	Debugging: error messages and common bugs
15	Complexity (Big-O basics)
16	Interview protocol: what to say and do, step by step
17	Flash-drill quiz with answers

1. How to attack ANY problem

Freezing happens when you try to write code before you understand the problem. Never start with code. Always run this checklist, out loud, in this order:

Step	Question to ask yourself	Pizza-style example
1. Restate	Say it in your own words in one sentence.	Given a size and toppings, compute the price.
2. Inputs	What do I receive? Types?	size (str), pepperoni (bool), extra_cheese (bool)
3. Output	What do I return or print? Type?	total price (number)
4. Rules	List every rule as a sentence.	size -> base price; pepperoni cost depends on size; cheese +1
5. By hand	Do one example manually, with numbers.	Medium + pepperoni = 15 + 3 = 18
6. Tools	Which Python tools? (table below)	variable, if/elif, +=
7. Pseudocode	Write 4-8 plain-English steps.	start total = base for size; add toppings; return total
8. Code	Translate each pseudocode line.	
9. Test	Run 3 cases: normal, edge, weird.	small/no toppings; large/all; invalid size
10. Debug	Print values, find the first wrong line.	

Tool picker: what in the English tells me what in Python?

If the requirement says...	Think...	Python
"if / otherwise / when / unless"	decision	if / elif / else
"for each / every / all / each item"	repeat over a collection	for x in items:
"until / as long as / keep doing"	repeat with a stop condition	while cond:
"total / sum / add up"	accumulator	total = 0; total += x
"count how many"	counter	count = 0; count += 1
"largest / smallest / best / cheapest"	track a best-so-far	best = items[0]; compare
"list of / collection / order matters"	list	nums = []
"look up by name/id / price of X / how many of each"	dictionary	d = {}; d[key]
"duplicate / unique / seen before / already exists"	set (or dict)	seen = set()
"calculate / reusable / do this many times"	function	def f(...): return ...
"at least / at most / more than / between"	comparison, mind the boundary	>= <= > <
"and / or / both / either"	boolean logic	and / or / not
"first N / last N / every other"	slicing / range step	nums[:N] nums[-N:] nums[::2]
"return / print / display / output"	return = give back; print = show	return x print(x)

Remember: If you freeze: answer 'What do I receive?' then 'What do I return?' then 'What would I do by hand with a small example?' Then convert each hand step into one line of code.

2. Program skeleton, input, output

Running code

```
# file: hello.py      run: python3 hello.py
print("Hello")        # shows text, then a newline
```

Input is ALWAYS a string. Convert it.

```
name = input("Name: ")      # str
age = int(input("Age: "))    # whole number
price = float(input("Price: ")) # decimal number
yes = input("Y/N? ").strip().lower() == "y" # clean + compare
```

Output

```
print("a", "b")              # a b          (space between)
print("a", "b", sep="-")     # a-b
print("no newline", end="")  # stays on same line
print(f"Total: {total}")     # f-string: put variables inside {}
print(f"Total: {total:.2f}") # 2 decimals -> 18.00
print(f"{x} + {y} = {x + y}") # expressions inside {} are fine
print("Total: " + str(total)) # + only joins str with str; convert first
```

Two ways to organize a program

```
# 1) Script style: lines run top to bottom
age = int(input("Age: "))
if age >= 18:
    print("ok")

# 2) Function style: define, then CALL it (a def alone does nothing!)
def main():
    age = int(input("Age: "))
    ...

main() # <- without this line, nothing runs

# Common professional pattern:
if __name__ == "__main__":
    main()
```

Remember: A function that is defined but never called prints nothing. This is the #1 beginner 'why no output' bug.

3. Variables, numbers, arithmetic, comparisons

Types

Type	Example	Notes
int	x = 5	whole numbers, unlimited size
float	x = 5.5	decimals; money rounding: round(x, 2)
str	s = "hi"	text; single or double quotes
bool	ok = True	True / False (capital T/F)
list	[1, 2, 3]	ordered, changeable
dict	{"a": 1}	key -> value
set	{1, 2, 3}	unique, unordered
tuple	(1, 2)	ordered, NOT changeable
None	x = None	'nothing'; default for 'no value yet'

```
type(5)    type(5.0)  type("5")    # check a type
int("42")  float("3.5") str(42)    # convert
int(3.9)   # 3 (cuts decimals) round(3.5) # 4    round(2.675, 2)
```

Arithmetic operators

Op	Meaning	Example	Result
+ - *	add subtract multiply	7 * 3	21
/	divide (always float)	7 / 2	3.5
//	floor divide (whole part)	7 // 2	3
%	remainder (modulo)	7 % 2	1
**	power	2 ** 5	32
x += 1	x = x + 1 (also -= *= /=)	total += price	

```
n % 2 == 0      # n is even          n % 2 == 1    # n is odd
n % 5 == 0      # divisible by 5
n // 10         # drop last digit    n % 10        # last digit
minutes = total_seconds // 60; secs = total_seconds % 60
abs(-5) min(3, 9) max(3, 9) sum([1, 2, 3]) round(7.456, 2)
import math; math.ceil(2.1) # 3 (round UP: e.g. pages needed, hours billed)
```

Comparison and boolean operators

==	equal	!=	not equal
>	greater	<	less
>=	greater or equal ('at least')	<=	less or equal ('at most')
and	both true	or	either true
not	flip true/false	in / not in	membership

```
18 <= age < 65      # chained: between (inclusive of 18, exclusive of 65)
age >= 18 and has_id # both needed
day == "Sat" or day == "Sun"
day in ("Sat", "Sun") # shorter way to test several values
not is_member
```

Remember: '18 or older' is `>= 18`. 'Over 18' is `> 18`. 'Under 18' is `< 18`. Underline the exact words in the requirement; boundary words decide `>=` versus `>`.

Assign, swap

```
a = b = 0           # both zero
a, b = 5, 9         # assign two at once
a, b = b, a         # SWAP (no temp variable needed)
x, y, z = [1, 2, 3] # unpack
```

Naming

Use descriptive snake_case: `total_price`, `item_count`, `is_member`. Constants in caps: `TAX_RATE = 0.08`. Never use a name like `list`, `str`, `sum`, `max` for your variables (it hides the built-in).

4. Strings

```
s = "Hello World"
len(s)           # 11
s[0] s[-1]       # 'H' 'd'   (first, last)
s[0:5] s[:5] s[6:] # 'Hello' 'Hello' 'World' [start:stop) stop excluded
s[::-1]          # reverse -> 'dlrow olleH'
s.lower() s.upper() s.title() s.strip() # strip removes spaces both ends
s.split()        # ['Hello', 'World']   split on whitespace
"a,b,c".split(",") # ['a', 'b', 'c']
"-".join(["a", "b"]) # 'a-b'         join list of str into one str
s.replace("l", "L") # 'HeLlo WorLd'
s.find("World")     # 6 (index) or -1 if missing
s.count("l")        # 3
s.startswith("He") s.endswith("d")      # True True
"World" in s        # True (substring check)
s.isdigit() s.isalpha() s.isalnum() s.isupper() s.islower() s.isspace()
for ch in s:        # loop over characters
    print(ch)
"a" * 3             # 'aaa'
"ab" + "cd"         # 'abcd'
```

Remember: Strings are immutable: `s.upper()` returns a NEW string. You must write `s = s.upper()`. You cannot do `s[0] = 'x'`.

Typical string tasks

```
# Palindrome (ignoring case)
t = s.lower(); is_pal = (t == t[::-1])

# Count vowels
count = 0
for ch in s.lower():
    if ch in "aeiou":
        count += 1

# Password checks
has_digit = any(ch.isdigit() for ch in pw)
has_upper = any(ch.isupper() for ch in pw)

# First letter of each word
initials = "".join(w[0] for w in s.split())
```

5. Conditions: if / elif / else

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
```

The rules that cause bugs

Rule	Why it matters
Order of elif matters: first True branch wins, rest skipped.	Check the most specific / highest first. 'score >= 70' before '>= 90' gives wrong grade.
Colon ':' after if/elif/else and indent the body (4 spaces).	Missing colon or bad indent = SyntaxError / IndentationError.
== compares, = assigns.	if x = 5 is an error. Write if x == 5.
Strings must match exactly, including case.	'Adult' != 'adult'. Use .lower() to compare ignoring case.
Separate if statements are ALL checked; elif chain stops at the first match.	Use if/elif/else for mutually exclusive choices; separate ifs for independent add-ons.
else has no condition.	It means 'everything not covered above'.

Mutually exclusive vs independent

```
# Exclusive: exactly ONE size, so if/elif/else
if size == "small": price = 10
elif size == "medium": price = 15
elif size == "large": price = 20
else: price = 0 # invalid size

# Independent: any combination of add-ons, so separate ifs
if extra_cheese: price += 1
if pepperoni: price += 2 if size == "small" else 3
```

Nested conditions and combining

```
if age >= 18:
    if has_ticket:
        print("Enter")
    else:
        print("Need ticket")
else:
    print("Too young")

# Same logic flattened:
if age >= 18 and has_ticket: print("Enter")
elif age >= 18: print("Need ticket")
else: print("Too young")
```

One-line conditional (ternary) and 'truthy' values

```
label = "Adult" if age >= 18 else "Minor"
fee = 0 if total > 50 else 7
# Falsy: 0, 0.0, "", [], {}, set(), None, False. Everything else is truthy.
if items: # True when the list is NOT empty
    ...
if not name: # True when name is empty string
```

...

Edge cases to ALWAYS test in condition problems

The exact boundary values (17, 18, 19 for an 18 rule); 0; negative; empty input; the largest tier; text with wrong capitalization; an unlisted option.

6. Loops

for loop: repeat over a collection or a range

```
for x in [4, 8, 15]:           # each item
    print(x)

for ch in "hey":               # each character
    print(ch)

for i in range(5):             # 0,1,2,3,4
for i in range(1, 6):          # 1..5 (stop is EXCLUDED)
for i in range(0, 10, 2):      # 0,2,4,6,8 (step)
for i in range(10, 0, -1):     # 10,9,...,1 (count down)
for i in range(len(nums)):     # indexes 0..len-1
    print(i, nums[i])
for i, x in enumerate(nums):   # index AND value together
    print(i, x)
for a, b in zip(names, ages):  # two lists in parallel
for k, v in d.items():         # dict key and value
```

while loop: repeat until a condition becomes False

```
count = 0
while count < 5:
    print(count)
    count += 1                # YOU must change something or it loops forever

while True:                   # 'run until I say stop'
    cmd = input("> ")
    if cmd == "quit":
        break
```

break, continue, else

```
for x in nums:
    if x < 0:
        continue            # skip this item, go to next
    if x == target:
        print("found")
        break                # leave the loop completely
else:
    print("not found")       # runs only if loop finished WITHOUT break
```

Choosing

Situation	Use
Know the collection or a count	for
Need the position (index)	for i in range(len(x)) or enumerate
Repeat until something happens (valid input, balance 0)	while
Don't need index	for x in items (cleanest)

Remember: Off-by-one: range(5) gives 0..4, range(1, 5) gives 1..4. range(len(nums)+1) goes one past the end and crashes with IndexError.

7. The 12 loop patterns (memorize the SHAPE)

Almost every beginner/medium problem is one of these shapes with different names. Practice by rewriting each with new variable names and new data (scores, prices, temperatures ...).

```
# 1. SUM (accumulator)
total = 0
for x in nums:
    total += x

# 2. COUNT items that satisfy a rule
count = 0
for x in nums:
    if x % 2 == 0:
        count += 1

# 3. AVERAGE (guard against empty!)
avg = total / len(nums) if nums else 0

# 4. MAX (track best so far)
best = nums[0]
for x in nums:
    if x > best:
        best = x
# MIN: same with <

# 5. MAX with its INDEX / position
best_i = 0
for i in range(len(nums)):
    if nums[i] > nums[best_i]:
        best_i = i

# 6. FILTER (build new list of matches)
big = []
for x in nums:
    if x > 50:
        big.append(x)

# 7. TRANSFORM (apply to every item)
doubled = []
for x in nums:
    doubled.append(x * 2)

# 8. SEARCH (does it exist? where?)
found = False
for x in nums:
    if x == target:
        found = True
        break

# 9. COUNT OCCURRENCES by key (frequency dict)
freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1

# 10. SEEN / DUPLICATE detection
seen = set()
dup = []
for x in nums:
    if x in seen:
        dup.append(x)
    seen.add(x)
```

```
# 11. TOTAL BY GROUP (dict accumulator)
by_cat = {}
for cat, amount in rows:
    by_cat[cat] = by_cat.get(cat, 0) + amount

# 12. WHILE until valid / until target
n = int(input("1-10: "))
while n < 1 or n > 10:
    n = int(input("Try again: "))
```

Shortcuts (use only after you can write the long form)

sum(nums)	min(nums)	max(nums)	len(nums)	sorted(nums)	any(...)	all(...)
[x * 2 for x in nums]				# transform		
[x for x in nums if x > 50]				# filter		
sum(1 for x in nums if x % 2 == 0)				# count matches		
any(x < 0 for x in nums)				# is any negative?		
all(x >= 0 for x in nums)				# are all non-negative?		

8. Lists

```
nums = [5, 3, 9]          empty = []          len(nums)      # 3
nums[0]  nums[2]  nums[-1]  nums[-2]         # first, third, LAST, 2nd last
nums[1] = 7               # change by index
nums[1:3]  nums[:2]  nums[1:]  nums[::-1]     # slices (stop excluded)
```

Task	Code	Notes
add to end	<code>nums.append(4)</code>	returns None
add at position	<code>nums.insert(0, 4)</code>	slow for big lists
add many	<code>nums.extend([1, 2])</code> or <code>nums += [1, 2]</code>	
remove last	<code>x = nums.pop()</code>	returns the removed item
remove at index	<code>nums.pop(i)</code> or <code>del nums[i]</code>	<code>IndexError</code> if bad i
remove by value	<code>nums.remove(9)</code>	first match; <code>ValueError</code> if missing
exists?	<code>9 in nums</code>	<code>True/False</code>
index of value	<code>nums.index(9)</code>	<code>ValueError</code> if missing
count value	<code>nums.count(9)</code>	
sort in place	<code>nums.sort()</code> / <code>nums.sort(reverse=True)</code>	returns None; changes list
sorted copy	<code>sorted(nums)</code> / <code>sorted(nums, reverse=True)</code>	original unchanged
reverse in place	<code>nums.reverse()</code>	returns None
reversed copy	<code>nums[::-1]</code>	
copy	<code>b = nums.copy()</code> or <code>b = nums[:]</code>	<code>b = nums</code> is NOT a copy
clear	<code>nums.clear()</code>	
length	<code>len(nums)</code>	
empty?	<code>if not nums:</code>	
join to string	<code>"-".join(["a", "b"])</code>	items must be str

Remember: `nums = nums.sort()` sets `nums` to `None`. `sort()`, `append()`, `reverse()` change the list and return `None`.

Three loop styles (know when to use each)

```
for x in nums:           # just values
for i in range(len(nums)): # need index (modify, compare neighbors)
for i, x in enumerate(nums): # need both
```

Modify while looping: safe vs unsafe

```
# Replace values: use the index
for i in range(len(nums)):
    if nums[i] < 0:
        nums[i] = 0

# Remove items: DON'T remove from the list you're looping over. Build a new one:
nums = [x for x in nums if x >= 0]

# Neighbor comparison: stop one early
for i in range(len(nums) - 1):
    if nums[i] > nums[i + 1]:
        print("not sorted")
```

More list recipes

```
# Second largest (distinct)
u = sorted(set(nums), reverse=True); second = u[1] if len(u) > 1 else None
```

```

# Remove duplicates, keep order
seen = set(); out = []
for x in nums:
    if x not in seen:
        seen.add(x); out.append(x)

# Reverse manually
rev = []
for i in range(len(nums) - 1, -1, -1):
    rev.append(nums[i])

# 2D list (grid)
grid = [[1, 2, 3], [4, 5, 6]]
grid[1][2]          # 6    row 1, col 2
for row in grid:
    for val in row: ...
rows, cols = len(grid), len(grid[0])

# Sort list of lists/tuples by a field
people = [("Ana", 30), ("Bo", 25)]
people.sort(key=lambda t: t[1])          # by age ascending
sorted(people, key=lambda t: t[1], reverse=True) # by age descending

```

Remember: Empty-list guard: `nums[0]`, `max(nums)`, `nums[-1]` all crash on `[]`. Ask 'what if empty?' before indexing.

9. Dictionaries

A dictionary maps a **key** to a **value** (name -> price, word -> count, id -> record). Use it whenever the English says 'look up', 'for each ___ keep a ___', or 'how many of each'.

```
prices = {"small": 10, "medium": 15, "large": 20}
empty = {}                                # NOT set()
prices["large"]                            # 20      KeyError if missing!
prices.get("huge")                         # None     safe
prices.get("huge", 0)                     # 0        safe with default
prices["xl"] = 25                          # add or overwrite
del prices["xl"]                           # remove (KeyError if missing)
prices.pop("xl", None)                     # remove safely
"small" in prices                          # True     checks KEYS
len(prices)

for k in prices:                           # keys
for v in prices.values():                   # values
for k, v in prices.items():                 # both
list(prices.keys()) list(prices.values()) list(prices.items())
max(prices, key=prices.get)                 # key with the largest value
sorted(prices.items(), key=lambda kv: kv[1], reverse=True) # by value desc
sorted(prices)                             # keys sorted
```

Core dictionary patterns

```
# Frequency count
freq = {}
for x in items:
    freq[x] = freq.get(x, 0) + 1
# equivalent:
if x in freq: freq[x] += 1
else:         freq[x] = 1

# Group items into lists
groups = {}
for name, dept in rows:
    if dept not in groups:
        groups[dept] = []
    groups[dept].append(name)

# Lookup table replaces long if/elif
BASE = {"small": 10, "medium": 15, "large": 20}
total = BASE[size]                # BASE.get(size) if size might be invalid

# Inventory
stock = {"apple": 5}
stock["apple"] -= 2                # sell 2
if stock.get("pear", 0) >= 1: ...  # in stock?

# Most common
best = max(freq, key=freq.get)

# Two Sum-style: remember what you've seen and where
index_of = {}
for i, x in enumerate(nums):
    need = target - x
    if need in index_of:
        print(index_of[need], i)
    index_of[x] = i
```

Remember: Keys must be unchangeable types (str, int, tuple). Lists can't be keys. A missing key with [] crashes; with .get() it doesn't.

Dictionary of dictionaries / list of dictionaries

```
students = [{"name": "Ana", "score": 91}, {"name": "Bo", "score": 78}]
for s in students:
    print(s["name"], s["score"])
best = max(students, key=lambda s: s["score"])
accounts = {"A1": {"pin": "1234", "balance": 100}}
accounts["A1"]["balance"] += 50
```

10. Sets and tuples

Set: unique items, instant 'have I seen it?'

```
seen = set()                # empty set (NOT {} which is a dict)
s = {1, 2, 3}
s.add(4)                    # add one (ignored if already there)
s.remove(4)                 # KeyError if missing
s.discard(4)               # safe remove
4 in s                      # True/False, FAST
len(s)
set([1, 1, 2])              # {1, 2}    remove duplicates
len(set(nums)) != len(nums) # does the list contain duplicates?
a | b                       # union      a & b  # intersection    a - b   # in a not in b
for x in s:                 # no order, no indexing (s[0] is an error)
```

Need	Use
Order or duplicates matter, indexing	list
Only 'is it present?' / unique values	set
A count or value attached to each item	dict

Tuple: fixed group of values

```
point = (3, 4)
x, y = point                # unpack
point[0]                    # 3 (read only; point[0] = 9 is an error)
def min_max(nums): return min(nums), max(nums) # returning two values = a tuple
lo, hi = min_max([3, 9, 1])
```

11. Functions

```
def calculate_total(price, quantity):    # parameters = the INPUTS
    total = price * quantity
    return total                        # return = the OUTPUT (hands value back)

result = calculate_total(4.5, 3)        # CALL it; arguments fill parameters
print(result)                          # 13.5
```

return vs print

	print(x)	return x
Purpose	show text on the screen	give a value back to the caller
Can you use it later?	No	Yes: $y = f(\dots)$; $y + 1$
Ends the function?	No	Yes, immediately
Default if absent	-	function returns None

Remember: If the requirement says 'return', do NOT print. If it says 'print', do NOT only return. Forgetting return makes the function give back None.

Features

```
def ship(total, member=False):        # default value; member is optional
    ...
ship(60) ship(60, True) ship(total=60, member=True)

def stats(nums):
    return min(nums), max(nums), sum(nums) / len(nums)  # several values
lo, hi, avg = stats([3, 5, 9])

def is_valid(pw):                     # boolean-returning helper; name starts with is_/has_
    return len(pw) >= 8 and any(c.isdigit() for c in pw)

def f(x):
    if x < 0:
        return "neg"                 # early return: handle special case first
    return "ok"
```

Variable scope

Variables created inside a function exist only there. Pass what you need in as parameters; return what you need out. Prefer returning a result over changing global variables.

Break a big problem into small functions

```
def calculate_subtotal(cart):    ...
def calculate_discount(subtotal, is_member):    ...
def calculate_tax(amount):      ...

def calculate_total(cart, is_member):
    sub = calculate_subtotal(cart)
    disc = calculate_discount(sub, is_member)
    taxed = calculate_tax(sub - disc)
    return round(sub - disc + taxed, 2)
```

Mutation gotcha

```
def add_one(nums):
    nums.append(1)           # CHANGES the caller's list
def add_one_safe(nums):
```

```
    return nums + [1]          # returns a new list, original untouched
def bad(items=[]):           # NEVER use a list as a default value
def good(items=None):
    if items is None: items = []
```

12. Rules-to-code patterns (pricing, billing, scoring)

These problems are all the same: **base amount + rules that add, subtract, or multiply**. Translate each English rule into one small piece of code.

English rule	Code shape
Price depends on a category	if/elif or lookup dict: PRICE[size]
Add \$X if option chosen	if option: total += X
Add \$X per unit / per hour	total += X * units
First N units at A, rest at B	total = A * min(n, N) + B * max(0, n - N)
Free if over \$50, else \$7	fee = 0 if total > 50 else 7
P% off / discount	total -= total * P / 100 (or total *= 1 - P/100)
Tax of P%	total += total * P / 100
Cap at \$20 (never more)	total = min(total, 20)
At least \$5 (minimum charge)	total = max(total, 5)
Tiered rates (0-100 at a, next 100 at b ...)	compute each band with min/max, then add
Different rules for different member types	dict of rates: RATE[member_type]
Round to cents	round(total, 2) print(f'{total:.2f}')
Charge by started hour (partial counts)	import math; math.ceil(minutes / 60)
Order of operations (discount before tax?)	Read the English carefully: do it in that order

Worked shapes (new contexts, learn the SHAPE)

```
# Parking: $5 first hour, $3 each extra hour, daily cap $20
def parking_fee(hours):
    if hours <= 0:
        return 0
    fee = 5 + 3 * (hours - 1)
    return min(fee, 20)

# Electricity tiers: first 100 units at 0.10, the rest at 0.15
def electric_bill(units):
    first = min(units, 100) * 0.10
    rest = max(units - 100, 0) * 0.15
    return round(first + rest, 2)

# Shipping: weight tiers, free over $50
def shipping(total, weight):
    if total > 50:
        return 0
    if weight <= 1: return 4
    elif weight <= 5: return 8
    else: return 12

# Membership discount with minimum order
def final_price(total, member):
    if total < 20:
        return total # no discount on small orders
    rate = {"premium": 0.20, "regular": 0.10}.get(member, 0)
    return round(total * (1 - rate), 2)
```

Remember: Write the rules as a numbered list in comments first. Then each comment becomes 1-3 lines of code. Test every boundary: exactly at the threshold, just below, just above.

13. Testing your code

```
# Quick manual testing: call it with known answers and print
print(parking_fee(1))      # expect 5
print(parking_fee(3))      # expect 11
print(parking_fee(10))     # expect 20 (capped)
print(parking_fee(0))      # expect 0 (edge)

# assert: crashes loudly if wrong, silent if right
assert parking_fee(1) == 5
assert parking_fee(10) == 20, "cap failed"
# floats: compare with round()
assert round(electric_bill(150), 2) == 17.5

# Table of cases
cases = [(1, 5), (2, 8), (6, 20), (10, 20), (0, 0)]
for hours, expected in cases:
    got = parking_fee(hours)
    print("OK " if got == expected else "BAD", hours, expected, got)
```

What to test (always 5 kinds)

Kind	Examples
Normal	a typical value
Boundary	exactly at 18, 50, 100 ... and one below and one above
Empty / zero	[], "", 0, no items
Single	list with one element
Odd / invalid	negative, unknown option, duplicates, wrong case

14. Debugging

Read the error from the BOTTOM up

```
Traceback (most recent call last):  
  File "ex.py", line 9, in <module>      <- where it happened (line 9)  
    print(nums[5])  
IndexError: list index out of range      <- WHAT happened (read this first)
```

Error	Usual cause	Check
SyntaxError	missing ':' or ')' or quote; = vs ==	the line above the reported line too
IndentationError	inconsistent spaces	use 4 spaces everywhere
NameError	variable not defined / typo / used before assignment	spelling, scope
TypeError	wrong type: 'a' + 5; calling int on list; wrong arg count	print(type(x)); convert with int()/str()
ValueError	int('abc'); remove() of missing value	validate input first
IndexError	index past end (off-by-one) or empty list	valid indexes: 0..len-1
KeyError	dict key missing	use .get(key, default) or check 'in'
ZeroDivisionError	divide by 0 / empty average	guard: if count == 0
AttributeError	method doesn't exist for that type; got None	often a variable became None
No output at all	function never called; wrong indentation; print missing	add a call / print
Infinite loop	while condition never becomes False	change the variable inside; Ctrl+C to stop

Logic-bug hunting method

1) Reproduce with the smallest input that fails. 2) Predict what each line should do. 3) `print(variable)` inside the loop to see the real values. 4) The bug is at the FIRST line where reality differs from your prediction. 5) Fix ONE thing, re-run.

Common bug checklist

Bug	Looks like	Fix
Off-by-one	<code>range(len(nums)+1)</code> / <code>range(1, n)</code>	recheck start and stop (stop excluded)
Wrong boundary	<code>></code> used where <code>>=</code> needed	re-read 'at least', 'over', 'up to'
Wrong variable	adds price instead of total	read names aloud
Forgot return	function result is None	add return
return inside loop too early	return in first iteration	move return after the loop
Accumulator inside loop	total = 0 inside the for	initialize BEFORE the loop
Print vs return	prints but returns None	match the requirement
Mutating while iterating	removing from list being looped	build a new list
Alias, not copy	<code>b = a</code> changes a too	<code>b = a.copy()</code>
<code>input()</code> is a string	<code>'5' + 1</code> -> TypeError; <code>'10' < '9'</code> is True	<code>int(input())</code>
Float precision	<code>0.1 + 0.2 != 0.3</code>	<code>round()</code> or compare with tolerance
Strings immutable	<code>s.upper()</code> didn't change s	<code>s = s.upper()</code>
Integer division	<code>7 / 2</code> gave 3.5, wanted 3	use <code>//</code>

15. Complexity (Big-O) basics

n = size of the input. Big-O says how the work grows as n grows. Interviewers always ask: 'What's the time and space complexity?'

Complexity	Name	Looks like
$O(1)$	constant	index a list, dict get/set, set add/in, append, pop()
$O(\log n)$	logarithmic	halving each step (binary search)
$O(n)$	linear	one loop over the input; sum, max, 'x in list'
$O(n \log n)$		sorting (sorted / .sort())
$O(n^2)$	quadratic	loop inside a loop over the same input
$O(2^n)$	exponential	trying all subsets

Operation	list	set / dict
x in collection	$O(n)$	$O(1)$ average
add at end	$O(1)$	$O(1)$
insert / remove at front or middle	$O(n)$	-
index by position	$O(1)$	lookup by key $O(1)$
sort	$O(n \log n)$	-

Space complexity: extra memory you use. A new list/dict/set sized by the input = $O(n)$. A few variables = $O(1)$.

Remember: Using a set or dict to replace a nested 'x in list' loop turns $O(n^2)$ into $O(n)$. This is the core idea behind many interview problems.

16. Interview protocol (what to SAY and DO)

When the interviewer says 'write the code', you are not allowed to go silent. Follow this script every time:

Step	What to say / do	Time
1	Repeat the problem in your own words.	20 s
2	Ask clarifying questions: types? negative values? empty input? duplicates? invalid input? what to return?	30 s
3	State inputs and output. Give one example with real numbers.	20 s
4	Say your plan in plain English: 'I'll loop through, keep a running total, and ...'	30 s
5	Write pseudocode comments in the editor.	1 min
6	Turn each comment into code. Talk while typing. Use clear names.	main time
7	Trace your code by hand with the example. Then test an edge case.	1-2 min
8	State time and space complexity.	20 s
9	Mention an improvement or alternative if you have one.	optional

If you freeze

Say: 'Let me do an example by hand first.' Then write what you did as numbered steps. Each step becomes a line or a loop. Start with a **brute-force, simple** solution that works. A correct simple solution beats an unfinished clever one.

Blank-file template you can type from memory

```
def solve(data):
    # 1. handle empty / edge case
    if not data:
        return ...
    # 2. set up variables (total, count, best, result list/dict/set)
    result = 0
    # 3. loop
    for item in data:
        # 4. decide (if/elif)
        if ...:
            result += ...
    # 5. return
    return result

# 6. test
print(solve([...]))    # expect ...
print(solve([]))      # expect ...
```

17. Flash-drill quiz (cover the right column)

Answer each from memory, out loud or on paper. Anything you miss goes into ERROR_LOG.md.

#	Question	Answer
1	Last element of a list nums?	<code>nums[-1]</code>
2	Index of the value 9 in nums?	<code>nums.index(9)</code>
3	Loop through values?	<code>for x in nums:</code>
4	Loop through indexes?	<code>for i in range(len(nums)):</code>
5	Index and value together?	<code>for i, x in enumerate(nums):</code>
6	Add an item to the end?	<code>nums.append(x)</code>
7	Remove the last item (and get it)?	<code>nums.pop()</code>
8	Remove the first occurrence of 5?	<code>nums.remove(5)</code>
9	Does 5 exist in the list?	<code>5 in nums</code>
10	Create an empty dictionary?	<code>d = {}</code>
11	Safely get a dict value, default 0?	<code>d.get(key, 0)</code>
12	Count frequencies?	<code>freq[x] = freq.get(x, 0) + 1</code>
13	Create an empty set?	<code>s = set()</code>
14	Add to a set?	<code>s.add(x)</code>
15	Sort ascending, in place?	<code>nums.sort()</code>
16	Sort descending, new list?	<code>sorted(nums, reverse=True)</code>
17	Reverse a string?	<code>s[::-1]</code>
18	Define a function?	<code>def name(params):</code>
19	Give a value back?	<code>return value</code>
20	Swap two variables?	<code>a, b = b, a</code>
21	Is n even?	<code>n % 2 == 0</code>
22	Convert input text to a whole number?	<code>int(input(...))</code>
23	Print with 2 decimals?	<code>print(f'{x:.2f}')</code>
24	Loop 1 to 10 inclusive?	<code>for i in range(1, 11):</code>
25	Loop 10 down to 1?	<code>for i in range(10, 0, -1):</code>
26	Largest of a list (built-in)?	<code>max(nums)</code>
27	Total of a list (built-in)?	<code>sum(nums)</code>
28	Number of items?	<code>len(nums)</code>
29	Check key in dict?	<code>key in d</code>
30	Loop dict keys and values?	<code>for k, v in d.items():</code>
31	First 3 items of a list?	<code>nums[:3]</code>
32	Every second item?	<code>nums[::2]</code>
33	Split 'a b c' into words?	<code>s.split()</code>
34	Join words with a space?	<code>' '.join(words)</code>
35	Cap a total at 20?	<code>total = min(total, 20)</code>
36	Remainder of 17 divided by 5?	<code>17 % 5 -> 2</code>
37	Whole-number division of 17 by 5?	<code>17 // 5 -> 3</code>
38	Remove duplicates from a list?	<code>list(set(nums))</code>

#	Question	Answer
39	Copy a list?	<code>nums.copy()</code> or <code>nums[:]</code>
40	Infinite loop with an exit?	<code>while True: ... break</code>